

↓ 기호는 앞선 논증과 연결되는 논증을, ↑ 기호는 앞선 논증을 설명하거나 뒷받침하는 논증을 의미한다.

1 프로그래밍 언어는 계산을 기술하기 위해 정의된 언어이다. 여기서 계산은 특정 값의 산출뿐 아니라, 컴퓨터가 수행할 수 있는 행위와 그 진행 과정을 포괄하는 넓은 의미로 사용한다.

1.1 (↓) 해당 언어로 작성되어 계산을 수행하는 단위가 되는 표현을 프로그램이라 부른다.

2 프로그래밍 언어의 명세라고 불리는 것은 적어도 해당 언어의 구문이나 의미의 전체 또는 일부를 정의한다고 생각한다.

2.1 (↑) 구문과 의미를 표현한다는 것은 어떤 형태와 조건을 만족하는 프로그램이 유효한 프로그램인지, 그리고 유효한 프로그램이 어떤 계산을 의미하는지를 이해하기 위해 필요한 정보를 표현하는 것이다.

2.2 (↓) 명세를 표현하는 방법은 다양하다. 자연어로 작성된 문서일 수도 있고, 형식적인 표기법으로 작성된 문서일 수도 있으며, 주어진 프로그램의 유효성이나 의미를 실제로 판단할 수 있는 실행 가능한 형태일 수도 있다.

2.3 (↓) 그러나 단순히 언어의 구문과 의미를 표현한다는 이유만으로 모든 표현을 명세라고 부르는 것은 어려워 보인다. 예를 들어 테스트 케이스는 몇 가지 경우에 대하여 언어의 동작을 보여줄 수 있지만, 그 자체로 해당 프로그래밍 언어의 일반적인 규칙을 기술한다고 보기는 어렵다고 생각된다.

3 하나의 프로그래밍 언어에는 여러 명세가 존재할 수 있다.

3.1 (↑) 언어 전체를 규정하기 위한 단 하나의 공식 명세가 존재할 수도 있지만, 특정 논문이나 도구가 언어의 일부분만을 대상으로 별도의 명세를 제공할 수도 있기 때문이다.

4 언어의 구문과 의미를 표현하는 것 중에 어떤 조건을 만족해야 명세인지는 모르겠다.

5 명세가 엄밀하다는 것은 명세에 적힌 표현의 의미가 혼동 없이 해석된다는 것이다.

5.1 (↑) 엄밀하다는 것은 반드시 프로그램의 결과가 하나로 결정된다는 뜻은 아니다. 예를 들어 여러 실행 결과를 허용하는 언어도 허용되는 결과의 범위가 명확하게 정의되어 있다면 엄밀하게 명세될 수 있다.

5.2 (↓) 프로그래밍 언어의 명세가 반드시 엄밀하진 않다. 자연어 표현의 의미가 문맥이나 독자의 판단에 따라 달라질 수 있는 경우처럼, 해석의 여지가 남아 있는 명세도 존재한다.

6 명세는 가능한 한 엄밀한 것이 바람직하다.

6.1 (↑) 명세의 엄밀성은 언어 규칙에 대한 공통된 이해를 형성하는 데 도움이 되고, 동일한 규칙을 서로 다르게 해석함으로써 발생하는 구현 간의 불일치를 줄일 수 있기 때문이다.

7 프로그래밍 언어의 개발과 유지보수에서는 명세를 엄밀하게 작성하는 것뿐만 아니라, 실제 실행을 가능하게 하는 구현체를 명세에 맞게 유지보수하는 것도 중요하다.

7.1 (↑) 프로그래밍 언어가 계산을 기술하기 위한 언어라면, 해당 언어가 실제로 사용되기 위해서는 프로그램이 의미하는 계산을 컴퓨터 위에서 수행할 수 있는 어떤 방법이 필요하기 때문이다.

7.2 (↓) 이때 명세와 구현체를 서로 독립적으로 작성하고 유지한다면, 언어의 의미가 변경될 때 그 변경을 구현에도 별도로 반영해야 하며, 구현이 변경된 명세와 여전히 일치하는지도 확인해야 한다.

8 명세의 기계화는 구조와 그 해석 방법이 이미 정해져 있는 입력 체계를 이용하여 명세를 기술하고, 기계가 그 구조를 이용하여 명세에 관한 처리를 수행할 수 있도록 만드는 과정이라고 생각된다.

8.1 (↑) 기계화를 단순히 “명세를 기계가 다룰 수 있는 형태로 만드는 일”이라고 정의하는 것은 부족하다. 이러한 정의라면 자연어 명세를 단순한 텍스트 파일

로 저장하는 것도 기계화에 포함된다. 그러나 이 경우 기계는 그 텍스트의 내부에서 무엇이 구문 규칙이고 무엇이 의미 규칙인지와 같은 명세의 구조를 알지 못한 채 문자들의 나열로만 다룰 수 있다.

8.2 (↓) 따라서 기계화 과정은 명세를 엄밀하게 만드는 한 방법이다.

9 기계화된 명세는 명세를 대상으로 하는 여러 기계적인 처리를 가능하게 한다.

9.1 (↑) 이러한 처리에는 실행기, 문서나 테스트 등의 산출물 생성, 명세로 작성한 언어에 대한 검사 및 분석, 메타적인 논증 등이 포함된다.

예를 들어 P4-SpecTec은 명세로부터 문서와 테스트 케이스를 생성하고, 프로그램을 실행하는 기능과 명세의 커버리지를 측정하는 기능을 제공한다. Twelf는 명세에 기술된 언어에 관한 메타적인 성질을 검증하는 기능을 제공한다.

9.2 (↓) 따라서 명세의 기계화는 프로그래밍 언어의 개발과 유지보수에 도움을 줄 수 있다. 기본적으로 명세의 변경 사항을 여러 산출물에 수동으로 반영하는 작업을 줄일 수 있고, 하나의 기계화된 명세를 여러 산출물이나 기능의 공통 입력으로 사용할 수 있는 경우에는 같은 정보를 수동으로 각각 반영하는 작업을 줄일 수 있기 때문이다.

10 기계화에서 명세를 작성하는 입력 체계는 중요하다.

10.1 (↑) 입력 체계는 기계화 도구의 사용자가 명세를 작성할 때 직접 사용하는 표면이며, 어떤 종류의 규칙과 정보를 명세에 표현할 수 있는지를 제한하기 때문이다.

10.2 (↑) 입력 체계가 표현할 수 있는 정보는 기계가 이후의 처리에서 이용할 수 있는 정보의 범위를 결정한다. 따라서 어떤 분석, 변환 또는 실행을 수행할 수 있는지는 입력 체계의 표현 능력에 의해 제약된다.

10.3 (↑) 다만 입력 체계가 충분한 정보를 표현할 수 있다고 해서 도구가 해당 기능을 실제로 제공하는 것은 아니다. 실제로 제공되는 기능은 입력 체계의 표현

능력과 도구가 구현한 처리 방법에 의해 함께 결정된다.

- 11 지금껏 조사에서는 프로그래밍 언어의 명세를 기계적으로 처리하는 것을 주요 지원 목적으로 두고 설계된 프레임워크를 정리하였다. 해당 도구가 프로그래밍 언어 이외의 다른 대상에도 적용될 수 있다는 이유로 제외하진 않았다.

11.1 (↑) 예를 들어 범용 프로그래밍 언어로 reference interpreter를 작성하는 것도 언어의 의미를 표현하는 한 방법일 수 있지만, 범용 프로그래밍 언어 자체는 프로그래밍 언어 명세의 특성을 고려하여 설계된 체계가 아니므로 이 서베이의 주요 조사 대상에서는 제외할 수 있었다.

11.2 (↑) Language workbench류의 도구 역시 동일한 기준으로 판단하였다. 단순히 언어의 구문을 정의하는 기능만을 제공하거나, 의미를 범용 프로그래밍 언어의 코드로 구현하도록 할 뿐 명세의 구조를 이용한 별도의 처리를 제공하지 않는 경우에는 조사 대상에서 제외하였다.

- 12 앞서 조사한 기계화 도구가 제공하는 여러 기능 중에서, 명세를 바탕으로 프로그램의 계산을 수행할 수 있게 하는 실행 능력에 특히 관심이 간다.

12.1 (↑) 여기서 실행 능력이란 명세 자체 또는 명세로부터 얻어진 실행 수단을 이용하여, 주어진 프로그래밍 언어의 프로그램이 의미하는 계산을 실제로 수행할 수 있게 하는 능력을 의미한다.

12.2 (↑) 실행 능력은 앞서 7에서 언급한 문제와 직접적으로 관련되기 때문이다.

12.3 (↓) 이러한 능력은 여러 방식으로 제공될 수 있다. 명세 자체가 직접 실행 가능할 수도 있고, 명세를 입력받아 프로그램을 실행하는 범용 실행 엔진을 제공할 수도 있으며, 명세로부터 인터프리터나 컴파일러와 같은 실행 코드를 생성할 수도 있다.

12.4 (↓) 또한 기계적인 절차로 실행 능력을 제공하는 경우, 실제로 실행 능력이 명세를 만족한다는 것을 보장할 수도 있다는 이점이 있다. 예를 들어, 인공지능을

이용하여 명세를 만족하는 구현체를 자동으로 생성할 수도 있겠으나, 7.2에서 언급된 문제가 해결되진 않는다.

13 따라서 이 서베이에서는 앞서 정리한 프로그래밍 언어 명세 기계화 도구 중에서 이러한 실행 능력을 제공하는 도구들을 조사하고 싶다.

13.1 (↓) 이를 조사하면 프로그래밍 언어 명세를 기계화하면서 실행 능력까지 얻고자 하는 사용자가 어떤 도구를 선택할 수 있는지, 그리고 각 도구가 어떤 종류의 명세를 요구하는지를 비교할 수 있을 것이다.

13.2 (↓) 또한 기존 도구가 사용하는 입력 체계와 실행 방법을 정리함으로써, 새로운 기계화 도구나 입력 체계를 설계할 때 고려해야 할 설계 요소를 제안할 수 있을 것이다.